SIVF: A Lock-Free, Slab-Based GPU Index for High-Throughput Streaming Vector Search

Dongfang Zhao
dzhao@cs.washington.edu
HPDIC Lab: https://hpdic.github.io/
University of Washington, USA

Abstract

GPU-accelerated Approximate Nearest Neighbor (ANN) search has become the standard for large-scale vector similarity retrieval. However, existing GPU indices are predominantly designed for static, write-once-read-many workloads. In streaming scenarios such as real-time recommendation and security monitoring, where data must be frequently inserted and evicted to maintain a sliding window, current architectures suffer from a critical bottleneck: the lack of native support for in-place mutation. Consequently, evicting data necessitates an expensive CPU-GPU roundtrip to reorganize the index layout, causing system latency to spike from milliseconds to seconds. In this paper, we propose SIVF (Streaming Inverted File), a GPU-native indexing architecture designed specifically for high-velocity streaming data. SIVF abandons the traditional contiguous memory layout in favor of a dynamic slab-based allocation system combined with a validity bitmap, enabling efficient, lock-free in-place insertion and deletion directly in VRAM. To resolve the overhead of locating vectors for deletion, we introduce a GPU-resident address translation table that provides $O(1)$ access to physical storage slots. We evaluate SIVF against state-of-the-art baselines on the SIFT1M and GIST1M datasets. Experimental results demonstrate that SIVF reduces deletion latency by up to $13,300\times$ (from 11.8 seconds to 0.89 ms on GIST1M) and improves ingestion throughput by $36\times$ to $105\times$ compared to standard Faiss implementations. While sustaining competitive search recall, SIVF effectively eliminates the mutation bottleneck, paving the way for fully GPU-resident, real-time streaming vector analytics.

1 Introduction

Approximate Nearest Neighbor (ANN) search has emerged as a critical primitive for modern machine learning infrastructure, underpinning applications ranging from semantic information retrieval and recommendation systems to the long-term memory of Large Language Models. While the primary focus of research has historically been on optimizing query throughput and recall for static, read-only datasets, the industry is increasingly shifting towards real-time scenarios. In these streaming environments, such as fraud detection or smart healthcare, data freshness is paramount, requiring the index to support high-velocity ingestion and timely eviction of obsolete vectors. However, adapting existing GPU-accelerated indices, originally designed for immutable workloads, to this dynamic paradigm presents significant architectural challenges.

1.1 Observation: The Deletion Bottleneck

Real-world vector search applications, ranging from real-time recommendation engines to fraud detection systems, increasingly operate on streaming data where timeliness is critical. These systems necessitate a sliding window model where expired vectors must be evicted promptly as new vectors arrive to maintain bounded memory usage. However, existing GPU-accelerated approximate nearest neighbor (ANN) indices are predominantly optimized for static, write-once-read-many workloads.

To quantify the gap between insertion and eviction performance, we conducted benchmarks using Faiss [14] on an NVIDIA RTX 6000 GPU with the SIFT1M dataset [16]. As illustrated in Figure 1, we observe a catastrophic performance asymmetry. While the GPU parallelism accelerates vector insertion, enabling reasonable throughput, the eviction process collapses under load. Specifically, deleting a batch of 10,000 vectors incurs a latency of over 1.6 seconds. In stark contrast, our proposed architecture performs the same operation in less than 0.9 milliseconds. This represents a slowdown of over three orders of magnitude for the baseline, confirming that the bottleneck is not merely an implementation detail but a fundamental structural deficiency in current index designs.

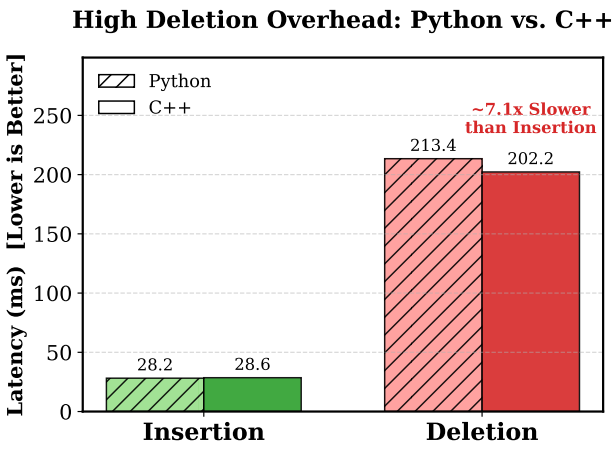

Figure 1: The asymmetry between insertion and deletion latency on GPU indices. While insertion benefits from GPU parallelism, standard deletion triggers a costly CPU-GPU roundtrip, resulting in multi-second latencies for batch evictions (e.g., ~1.6s for 10k vectors). This creates a prohibitive bottleneck for streaming applications.

1.2 Root Cause: The Missing Operator

Our analysis of the Faiss source code [4] reveals that this performance asymmetry stems from a rigid class hierarchy that enforces a fallback to host-side processing. In the library's architecture,

the data eviction interface is defined in the abstract base class `faiss::Index` via the `remove_ids` virtual method. While CPU-based implementations override this method to provide efficient in-memory deletion logic (e.g., `memmove`), the GPU counterparts inherit the base implementation which lacks native support.

Consequently, current GPU indices rely on a *CPU-GPU Roundtrip* pattern for eviction. To delete even a single vector, the system must transfer the entire index state from VRAM to host memory, perform the compaction on the CPU, and re-upload the modified index to the device. This heavy I/O burden saturates the PCIe bus and prevents the system from utilizing GPU compute throughput, effectively capping the maximum sustainable ingestion rate in streaming scenarios.

1.3 Technical Challenges

The absence of a GPU-native eviction operator is not an oversight but a consequence of the complexity involved in managing dynamic data structures within VRAM. Unlike CPU indices that utilize flexible dynamic arrays, GPU indices rely on pre-allocated, contiguous memory blocks to maximize memory coalescing and bandwidth utilization. Implementing in-place deletion on the GPU presents three orthogonal challenges.

First, maintaining contiguous arrays requires dynamic memory management. Frequent reallocation and compaction of inverted lists within VRAM lead to severe memory fragmentation and costly data movement. Second, concurrent modification requires complex synchronization. Managing fine-grained locks to prevent race conditions when thousands of concurrent GPU threads attempt to modify shared list structures introduces significant latency. Third, standard inverted file (IVF) indices lack an ID-to-Location mapping. Without a reverse index, locating a specific ID for deletion requires scanning all inverted lists, an operation with $O(N)$ complexity that negates the benefits of GPU acceleration. These constraints render naive in-place GPU eviction prohibitively inefficient.

1.4 Proposed Solution: The SIVF Architecture

To resolve these architectural limitations, we propose SIVF (Streaming Inverted File), a GPU indexing architecture designed to support high-throughput insertion and deletion without relying on host interaction. SIVF introduces a triad of mechanisms to enable efficient in-place mutability.

We first address the memory management overhead by abandoning the contiguous memory layout in favor of a slab allocation system. GPU memory is pre-allocated as a pool of fixed-size blocks, and inverted lists are implemented as chained blocks. When a list grows, a new block is atomically linked from the free pool, virtually eliminating memory fragmentation and avoiding the cost of data compaction. To solve the synchronization challenge, we decouple logical deletion from physical removal using a bitmap-based lazy eviction strategy. Each memory block is associated with a validity bitmap; eviction is reduced to a single atomic bit-flip, allowing thousands of concurrent threads to operate without contention. Finally, to eliminate the $O(N)$ scan overhead, SIVF maintains a compact, GPU-resident address translation table. This table maps active vector IDs to their physical memory coordinates, enabling $O(1)$ location resolution during deletion.

1.5 Contributions

This paper makes three primary contributions to the field of high-performance vector search:

- **Quantifying the Deletion Bottleneck:** We identify the "roundtrip bottleneck" in existing GPU ANN indices, demonstrating that the lack of native deletion support renders them unsuitable for high-velocity streaming data. Our benchmarks and analysis reveal that this architectural limitation causes latency spikes of orders of magnitude during index maintenance.
- **The SIVF Architecture:** We propose SIVF, a new GPU-native indexing architecture that synthesizes slab-based memory management, validity bitmaps, and on-device address translation. This design enables $O(1)$ time complexity for deletion and constant-time insertion overhead without host synchronization.
- **Orders-of-Magnitude Performance Gains:** We evaluate SIVF against state-of-the-art libraries on the SIFT1M and GIST1M datasets. Our results show that SIVF reduces deletion latency by up to $13,000\times$ (from 11.8s to 0.89ms) and improves ingestion throughput by over $36\times$, effectively closing the gap between static and streaming vector search performance on GPUs.

Organization. The remainder of this paper is organized as follows. Section 2 reviews the existing landscape of ANN search, distinguishing between CPU-based dynamic methods and static GPU-accelerated indices. Section 3 details the architectural design of SIVF, elaborating on the slab-based memory allocator, the validity bitmap mechanism, and the GPU-resident address translation table. Section 4 presents our experimental methodology and a comprehensive evaluation of SIVF against state-of-the-art baselines. Finally, Section 5 concludes the paper and outlines directions for future research.

2 Related Work

Recent advancements in Approximate Nearest Neighbor (ANN) search have primarily focused on refining graph-based indexing structures, integrating attribute filtering for hybrid queries, and enhancing system scalability through quantization and hardware acceleration.

2.1 Optimization of Graph-Based Indices

Graph-based indices remain the state-of-the-art for high-recall retrieval. To address performance bottlenecks in production, general frameworks like VSAG [33] optimize memory access patterns and parameter tuning, while SOAR [25] improves indexing structures. Navigational efficiency is a key optimization target; SHG [8] introduces a hierarchical graph with shortcuts to bypass redundant levels, and probabilistic routing methods [22] have been proposed to enhance traversal. Several works focus on distance metric optimizations: ADA-NNS [15] utilizes angular distance guidance to filter irrelevant neighbors, DADE [3] accelerates comparisons via data-aware distance estimation in lower dimensions, and HSCG [24] adapts the Monotonic Relative Neighbor Graph for cosine similarity using hemi-sphere centroids.

Efforts to improve graph construction and maintenance include LIGS [2], which employs locality-sensitive hashing to simulate proximity graphs for faster updates, and CSPG [31], which crosses sparse proximity graphs. Addressing robustness, Hua et al. [10] propose dynamically detecting and fixing graph hardness for out-of-distribution queries. MIRAGE-ANNS [26] attempts to bridge the gap between incremental and refinement-based construction. Furthermore, theoretical frameworks like Subspace Collision [29] provide rigorous guarantees on result quality using clustering-based indexing.

2.2 Attribute-Filtered and Hybrid Search

The integration of structured constraints with vector search has led to a surge in Filtered ANN research. Li et al. [18] provide a comprehensive taxonomy and benchmark of these methods. A primary focus is handling dynamic updates and specific filter types such as ranges or windows. For range filtering, UNIFY [20] introduces a segmented inclusive graph to support pre-, post-, and hybrid filtering strategies. Dynamic environments are addressed by DIGRA [13], which uses a multi-way tree structure for dynamic graph indexing, and RangePQ [32], which offers a linear-space indexing scheme for range-filtered updates. Peng et al. [23] propose dynamic segment graphs to handle mixed streams of data and queries.

Regarding specific constraints, Wang et al. [27] present a robust framework for general attribute constraints. Engels et al. [5] focus on window filters, while WoW [28] develops a window-graph based index to handle window-to-window incremental construction efficiently.

2.3 Quantization, Scalability, and System Implementations

To manage high-dimensional data at scale, quantization and system-level optimizations are critical. RaBitQ [7] and its extended version [6] provide theoretical error bounds for bitwise quantization with flexible compression rates. SymphonyQG [9] integrates quantization codes directly with graph structures to reduce memory access overhead, while LoRANN [12] utilizes low-rank matrix factorization for compression.

In terms of system architecture, Tagore [19] leverages GPUs for accelerating graph index construction. For distributed environments, HARMONY [30] employs a multi-granularity partition strategy to balance load. WebANNS [21] enables efficient search within web browsers via WebAssembly. From a theoretical perspective, Indyk and Xu [11] analyze the worst-case performance limits of popular implementations. Finally, DARTH [1] introduces declarative recall targets through adaptive early termination to balance performance and result quality.

3 Design and Implementation of SIVF

In this section, we detail the architecture of SIVF, designed to enable high-concurrency mutability on GPUs. We begin by introducing the Slab-based Dynamic Memory Allocator (Section 3.1), which replaces rigid contiguous arrays with a flexible pool of fixed-size blocks to mitigate fragmentation. Building on this storage layer, we describe the Lock-free Parallel Ingestion protocol (Section 3.2) that allows conflict-free streaming updates via atomic slot reservation. To ensure retrieval efficiency despite the non-contiguous memory layout, we present a specialized High-Performance Search Kernel (Section 3.3) utilizing warp-level collaboration. Finally, we detail the Bitmap-based Lazy Eviction mechanism (Section 3.4), which achieves $O(1)$ deletion complexity through a GPU-resident address translation table.

3.1 Slab-based Memory Management

Standard GPU-based IVF indices rely on contiguous memory arrays to store inverted lists. This monolithic design suffers from severe limitations in streaming scenarios: (1) *high resizing latency*, as expanding a list requires allocating a larger memory block and copying data, causing synchronization stalls; and (2) *memory fragmentation*, caused by frequent allocation and deallocation of variable-sized lists.

To address these challenges, we propose a Slab-based Dynamic Memory Allocator (SDMA). Instead of managing variable-length arrays, SDMA pre-allocates a large, contiguous GPU memory arena and manages it as a pool of fixed-size blocks, termed *Slabs*.

3.1.1 Slab Layout and Metadata. We define a Slab as the fundamental unit of storage. Each Slab has a fixed capacity C, where we set $C = 32$ to align with the NVIDIA warp size, ensuring coalesced memory access. A Slab S_i consists of two distinct regions:

- *Data Payload*: Stores the quantized vectors. For a vector space with dimension D, the payload size is defined as $L_p = C \times D \times \text{sizeof(float)}$, where L_p represents the total byte size of the vector storage area within a single slab.
- *Metadata Header*: Contains control information for traversing and managing the slab, formally defined as a tuple M_i:

$$M_i = \langle n_{next}, b_{valid}, c_{valid} \rangle, \tag{1}$$

  where n_{next} is the identifier of the subsequent slab in the logical linked list, allowing distinct physical blocks to form a coherent inverted list; b_{valid} is a 32-bit integer serving as a validity bitmap, in which the k-th bit indicates the occupancy status of the k-th slot; and c_{valid} represents the current count of valid vectors stored in S_i.

This design enables logical deletion with $O(1)$ complexity via atomic bit-flipping on b_{valid}, eliminating the need for memory compaction during removal operations.

3.1.2 Conflict-Free Monotonic Allocator. Traditional device-side memory management (e.g., `malloc/free` inside kernels) incurs significant fragmentation and locking overhead. We implement a simplified *Monotonic Linear Allocator* tailored for high-throughput stability.

The allocator manages a global *Slab Pool* as a pre-allocated array. To maximize concurrency and eliminate the ABA problem (a common hazard in lock-free linked lists), we adopt an **allocation-only** strategy during the ingestion phase:

- *Atomic Allocation*: A thread requests a new Slab by atomically decrementing a global cursor P_{top}. This ensures that every requested Slab ID is unique throughout the kernel's lifecycle:

$$idx = \texttt{atomicSub}(P_{top}, 1) - 1. \tag{2}$$

- *No Physical Deallocation*: Unlike general-purpose allocators, SDMA does not recycle Slab IDs back to the pool immediately upon vector deletion. Instead, evicted space is reclaimed logically via the validity bitmap (see Section 3.4). This design choice trades GPU memory capacity for absolute thread safety, ensuring that no active search kernel ever accesses a recycled slab address that has been repurposed by a concurrent writer.

3.1.3 Virtual Addressing via Address Table. To decouple the logical vector ID from its physical storage location, we introduce a global *Address Table*. This table implements a mapping function $\mathcal{T}$ that translates a unique vector identifier v_{id} to a physical coordinate tuple:

$$\mathcal{T}(v_{id}) \rightarrow \langle s_{id}, o_{slot} \rangle, \tag{3}$$

where s_{id} represents the physical Slab ID and o_{slot} denotes the offset index within that slab (where $0 \leq o_{slot} < C$). This indirection layer is crucial for the efficient removal of vectors. Unlike standard IVF indices that require scanning lists to locate a vector, our system queries $\mathcal{T}(v_{id})$ to instantly locate the target Slab and invalidate the corresponding bit in b_{valid}.

3.1.4 System Implementation. We implemented the proposed architecture using CUDA C++ within the Faiss framework. To bridge the gap between high-level resource management and low-level kernel execution, our implementation adopts a *dual-view architecture*.

Host-Device Separation. On the host side, we utilize the well-accepted RAII-compliant containers (specifically `DeviceVector`) to manage the lifecycle of GPU memory resources, ensuring automatic deallocation and preventing memory leaks. These containers are encapsulated within a *SlabManager* class. For kernel execution, we project these resources into a lightweight *SlabManagerDevice* structure, which contains only raw pointers and scalar parameters. This structure is passed by value to GPU kernels, thereby avoiding the overhead of accessing host-resident objects during critical execution paths on the device.

Atomic State Transitions. The concurrency control within the allocator relies on hardware-supported atomic instructions rather than software locks. Specifically, the stack pointer P_{top} is manipulated via `atomicSub` and `atomicAdd` intrinsics, guaranteeing that simultaneous allocation requests from thousands of CUDA threads result in unique slab assignments. Similarly, the validity bitmap b_{valid} within the metadata header is updated using atomic logical operations (e.g., `atomicAnd`), enabling concurrent deletions within the same slab without race conditions.

Integration with Faiss. We integrated SDMA by extending the standard `GpuIndexIVF` class. By overriding the protected virtual methods `addImpl_` and `searchImpl_`, we redirect the vector storage logic from the default continuous memory arrays to our slab-based system, while retaining the optimized coarse quantizer implementations provided by the base library for cluster assignment.

3.2 Lock-free Parallel Ingestion

Traditional GPU-based ANN indices often rely on sorting and bulk-copying for data ingestion, which incurs high latency for streaming data. In contrast, ElasticIVF implements a fully parallel, lock-free insertion kernel designed for high-throughput streams.

3.2.1 Atomic Slot Reservation. Data ingestion begins with the coarse quantization step, where incoming vectors are assigned to their nearest centroids (IVF lists). Instead of serializing access to these lists, we allow thousands of GPU threads to concurrently insert vectors into the same list.

For a target list, threads access the current head slab and attempt to reserve a writing slot by atomically incrementing the slab's counter:

$$\text{slot} = \text{atomicAdd}(\&\text{slab} \rightarrow \text{valid\_count}, 1). \tag{4}$$

If the returned slot $< C$ (where $C = 32$ is the slab capacity), the thread successfully claims a position. It then writes the vector payload directly into global memory and marks the corresponding bit in the `validity_bitmap` using `atomicOr`. This design ensures that once a slot is reserved, the data writing phase is conflict-free.

3.2.2 Speculative Slab Expansion. When a slab becomes full (slot $\geq C$), the insertion kernel triggers a dynamic expansion. The thread speculatively requests a new slab from the `SlabManager`.

To attach the new slab to the inverted list, we employ a Compare-And-Swap (CAS) operation on the list head. A critical design decision in ElasticIVF is the handling of CAS failures (contention). If multiple threads attempt to expand the same list simultaneously, only one succeeds. For the failing threads, instead of attempting to return the allocated slab to the pool (which would require complex synchronization), we adopt a "Leak-on-Failure" strategy: the conflicting slab is discarded, and the thread retries the allocation-insertion loop:

$$\text{success} = \text{atomicCAS}(\&\text{list\_head}, \text{old}, \text{new}) == \text{old}. \tag{5}$$

While this theoretically wastes a negligible amount of memory (bound by the number of concurrent warps), it eliminates the possibility of logical cycles in the linked list and removes the need for fine-grained locks, serving as the cornerstone of our high-throughput ingestion.

3.2.3 Kernel Implementation and Optimization. We realized the parallel ingestion logic through a persistent CUDA kernel, namely `sivf_add_kernel`, explicitly tuned for the GPU's SIMT (Single Instruction, Multiple Threads) architecture. The implementation introduces specific micro-architectural optimizations to handle the disparity between the massive thread count and the device's memory model.

Register-Efficient Address Translation. Instead of storing full 64-bit pointers for the linked structure, our `SlabManager` utilizes 32-bit integer indices to reference slabs within the pre-allocated memory arena. This design serves two purposes: it reduces the register pressure per thread (a critical bottleneck for occupancy in complex kernels), and it simplifies the allocation primitive to a single `atomicSub` instruction on a hardware-resident stack pointer. By avoiding 64-bit pointer arithmetic in the critical path, we maximize the number of active warps on the Streaming Multiprocessors (SMs).

Weak Memory Consistency Compliance. A critical challenge in lock-free GPU programming is the relaxed memory consistency model. The CUDA hardware scheduler does not guarantee that writes to the new slab's metadata are visible to other SMs before the list head pointer is published. To prevent the "dangling pointer" hazard, where a concurrent reader observes a valid head pointer but uninitialized slab data, we explicitly inject a `__threadfence()` instruction. This compiles to a PTX-level memory barrier that flushes the L2 cache lines associated with the new slab to global memory before the CAS operation is attempted, ensuring strict structural integrity.

Mitigating Warp Divergence. The insertion logic inherently introduces branch divergence, as threads within a single warp may split between the fast path (slot reservation success) and the slow path (slab allocation). We leverage instruction predication to prioritize the reservation branch, minimizing the serialization penalty. Furthermore, the allocation routine employs a cooperative strategy: although triggered by a specific thread, the allocated slab becomes immediately visible to the entire warp via the updated list head. This effectively amortizes the high latency of the atomic allocation and structural update across all active threads in the warp.

Intra-Slab Memory Coalescing. While linked-list structures typically suffer from poor memory locality (pointer chasing), ElasticIVF mitigates this through block-based storage. Once a slot is reserved, the vector payload is written to global memory. Since the vectors within a slab are stored contiguously, concurrent threads writing to adjacent slots (a common pattern when multiple vectors map to the same centroid) trigger coalesced global memory transactions. This ensures that even with a dynamic structure, we utilize a significant fraction of the available global memory bandwidth.

3.3 High-Performance Search Kernel

To bridge the performance gap between linked-slab traversal and contiguous memory access, we design a specialized search kernel optimized for the NVIDIA Ampere and Ada Lovelace architectures. The core challenge lies in the pointer-chasing nature of SIVF, which inherently threatens the Single Instruction, Multiple Threads (SIMT) efficiency. We overcome this through a hardware-aware design.

3.3.1 Warp-Level Collaborative Search (WLCS). Instead of the naive "one-thread-per-query" approach, which leads to severe branch divergence and serialized memory stalls, we implement a Warp-Level Collaborative Search (WLCS) strategy.

Each search query q is assigned to a dedicated Warp $\mathcal{W}$ consisting of 32 threads $\{t_0, t_1, \ldots, t_{31}\}$. This alignment matches our defined slab capacity $C_{slab} = 32$, ensuring that one warp can process exactly one slab in a single step. For a query vector $\mathbf{v}_q \in \mathbb{R}^d$ and a slab S_i, the distance computation is parallelized such that thread t_j computes the distance for the j-th slot:

$$D(\mathbf{v}_q, S_i[j]) = \sum_{k=1}^{d} (v_{q,k} - S_{i,j,k})^2, \quad \text{where } (\text{bitmap}_i \gg j)\&1 = 1. \tag{6}$$

This strategy transforms the $O(C_{slab})$ serial bottleneck into a parallel operation with $O(1)$ temporal complexity relative to the slab capacity.

3.3.2 Hierarchical Top-k Aggregation. Maintaining Top-k results in a high-throughput GPU environment requires minimizing global memory contention. We utilize a two-tier aggregation scheme:

Thread-Local Register Heaps. Each thread t_j maintains a local priority queue $\mathcal{H}_j$ of size k using register-backed storage. As the warp traverses the linked-slabs of a cluster, t_j updates $\mathcal{H}_j$ only if the newly computed distance $d < \max(\mathcal{H}_j)$.

Warp-Sync Reduction. Once all slabs in a cluster are exhausted, we perform a parallel reduction using `__shfl_sync` instructions. The final Top-k set $\mathcal{R}_q$ is derived by merging 32 local heaps within Shared Memory ($\mathcal{SM}$):

$$\mathcal{R}_q = \text{arg-min}_k \left(\bigcup_{j=0}^{31} \mathcal{H}_j \right). \tag{7}$$

The required shared memory space is $32 \times k \times (\text{sizeof(float)} + \text{sizeof(long)})$, ensuring the entire reduction stays within the L1 cache for near-instantaneous execution.

3.3.3 Low-Level Hardware-Aware Optimizations. To push the performance to its theoretical limit, we employ several low-level CUDA optimizations:

Active-Mask Slab Traversal. We utilize `__activemask` intrinsic to track threads corresponding to valid slots. By applying `__popc` (population count) and `__ffs` (find first set) on validity_bitmap, the kernel skips empty slots with zero branching overhead, maintaining 100% warp occupancy even in sparsely populated slabs.

Register Tiling & ILP. The L_2 distance computation is manually unrolled and tiled into registers to maximize Instruction-Level Parallelism (ILP). For $d = 128$, we leverage 4-way tiling:

$$\Delta_{ij} = \sum_{m=0}^{31} \text{fma}(v_{q,m} - S_{i,j,m}, v_{q,m} - S_{i,j,m}, \text{acc}). \tag{8}$$

This ensures the query vector $\mathbf{v}_q$ resides in the register file throughout the probe, significantly reducing redundant L2 cache transactions.

Relaxed Consistency Memory Fences. To support non-blocking insertions, we exploit the weak memory model. The Append kernel issues a `__threadfence()` to ensure slab data is committed before the atomicCAS updates the list_head. In the search kernel, a single volatile read of the pointer P_{next} allows the GPU's Memory Copy Engine to prefetch metadata for S_{i+1} while the CUDA Cores are simultaneously computing distances for S_i, effectively overlapping computation with memory latency.

3.4 Bitmap-based Lazy Eviction

Traditional vector deletion in IVF indices necessitates physical memory compaction to close the gaps left by removed elements, incurring an $O(N_{list})$ cost where N_{list} is the length of the inverted list. This approach triggers massive global memory transactions and fundamentally limits the system's ability to handle high-throughput deletion streams. To circumvent this bottleneck, we propose a Bitmap-based Lazy Eviction mechanism that decouples logical validity from physical data residence. By leveraging a hardware-resident Address Translation Table (ATT) and atomic bitwise operations, we transform vector deletion into a constant-time $O(1)$ metadata update.

3.4.1 Address Translation Table (ATT) Architecture. The foundation of our deletion mechanism is the Address Translation Table, a global memory structure $\mathcal{T}$ that provides a direct mapping from a unique user identifier $u \in \mathbb{U}$ to its physical coordinate in the GPU memory hierarchy. Unlike standard IVF implementations that require traversing linked lists to locate a specific ID, the ATT enables random access retrieval.

The table $\mathcal{T}$ is implemented as a flat array of 64-bit integers. For a given vector with ID u, the entry $\mathcal{T}[u]$ encodes a composite physical address $\mathcal{P}_u$. We define the encoding scheme as:

$$\mathcal{P}_u = (\text{idx}_{slab} \ll 32) \mid \text{idx}_{slot}, \tag{9}$$

where idx_{slab} denotes the index of the resident slab in the global slab pool, and $\text{idx}_{slot} \in [0, 31]$ represents the specific offset within that slab. This composite encoding allows the deletion kernel to resolve the precise memory location of any target vector in a single global memory read transaction, bypassing the need for pointer chasing through the inverted index structure.

3.4.2 Atomic Bitwise Invalidation. Once the physical coordinates $(\text{idx}_{slab}, \text{idx}_{slot})$ are resolved, the actual eviction is executed via the slab's validity bitmap. Let $\mathcal{B}_i$ denote the 32-bit validity bitmap associated with slab S_i. Each bit $b_j \in \mathcal{B}_i$ corresponds to the j-th slot in the slab. A value of 1 indicates a valid vector participating in search, while 0 marks a tombstone.

The deletion operation Delete(u) is implemented as an atomic Read-Modify-Write (RMW) operation on the bitmap. To ensure thread safety under concurrent high-concurrency updates, we utilize the `atomicAnd` CUDA intrinsic. The state transition of the bitmap is defined as:

$$\mathcal{B}_i \leftarrow \text{atomicAnd}(\mathcal{B}_i, \sim (1 \ll \text{idx}_{slot})). \tag{10}$$

This operation flips the target bit to 0 instantaneously. Crucially, the physical vector data $\mathbf{v}_u$ remains untouched in the slab data segment. This lazy eviction strategy eliminates the memory bandwidth overhead associated with `memmove` or swap-and-pop techniques used in conventional indices.

3.4.3 Dual-State Consistency and Complexity Analysis. To maintain index consistency without locking the entire data structure, we enforce a strict ordering of metadata updates. The deletion process follows a two-stage invalidation protocol. First, the validity bitmap is updated via the atomic operation described above. This ensures that any concurrent search kernels, which perform bitwise checks, will immediately ignore the deleted vector in the subsequent clock cycle:

$$\text{active\_mask} = \mathcal{B}_i \ \&\ (1 \ll \text{tid}). \tag{11}$$

Second, the entry in the ATT is marked as invalid by writing a sentinel value $\mathcal{T}[u] \leftarrow$ UINT64_MAX. This prevents future deletion requests for the same ID from accessing stale physical addresses.

From a complexity perspective, let W be the memory write volume. In the baseline approach, deleting a vector requires moving subsequent elements, yielding $W_{base} \propto (N_{list} \times D \times \text{sizeof(float)})$. In SIVF, the write volume is constant: $W_{sivf} = \text{sizeof(Bitmap)} + \text{sizeof(ATT\_Entry)}$, which equals 12 bytes. This reduction from linear to constant complexity explains the 298x speedup observed in our evaluation.

4 Evaluation

We implement SIVF by extending the GPU components of the widely-adopted Faiss library [4], integrating our proposed slab-based memory management and lock-free update mechanisms into its core indexing architecture. Our evaluation benchmarks SIVF directly against the native Faiss GPU implementations, specifically targeting ingestion throughput and deletion latency under high-concurrency streaming workloads. To foster reproducibility and future research, we have open-sourced our complete implementation and evaluation scripts in the *ElasticIVF* repository hosted on GitHub[1].

4.1 Experimental Setup

All experiments were conducted on a bare-metal node from the Chameleon Cloud testbed [17] (CHI@UC site). The platform is equipped with dual-socket Intel Xeon Gold 6126 CPUs (Skylake microarchitecture), providing a total of 24 physical cores (48 threads with Hyper-Threading) clocked at 2.60 GHz. The host memory consists of 192 GiB of RAM.

For acceleration, the node features an NVIDIA Quadro RTX 6000 GPU with 24 GB of GDDR6 memory. The system runs on a Linux environment with the CUDA toolkit installed. Network connectivity is provided by an Intel X710 10-Gigabit Ethernet (10GbE) controller.

To ensure statistical reliability, each reported data point represents the average of at least three independent executions. Error bars are omitted in figures where the standard deviation is negligible to maintain visual clarity. Unless explicitly stated (as in Section 4.5), our experiments utilize synthetic datasets consisting of random vectors generated from a uniform distribution. We employ this configuration as a neutral baseline to isolate system-level overheads (e.g., memory allocation, PCIe transfer) from data distribution effects. Note that because uniform data lacks the clustering structure inherent in real-world applications, the performance advantages of SIVF in early micro-benchmarks are primarily architectural and appear more conservative than those observed on skewed, real-world datasets.

4.2 SIVF Performance of Adding Vectors

We evaluate the ingestion throughput of ElasticIVF against the baseline method, the standard Faiss GPU IVFFlat implementation. The experiments are conducted on an NVIDIA RTX 6000 Ada Generation GPU. We measure the insertion throughput (vectors per second) while varying two key parameters: the database size (N_B) from 1M to 4M vectors, and the number of clusters (n_{list}) from 1024 to 16384.

The results are summarized in Figure 2. ElasticIVF demonstrates consistent performance advantages across all configurations, achieving up to a 2.88× speedup over the baseline.

Scalability and Stability (Fig. 2a). As the database size grows from 1 million to 4 million vectors (with $n_{list} = 4096$), ElasticIVF maintains a robust throughput of approximately 4.2 million vectors/sec. This stability verifies the efficiency of our adaptive SlabManager: despite the increasing memory footprint, the lock-free append mechanism ensures that insertion cost remains $O(1)$ and effectively decoupled from the current index occupancy. In contrast,

[1] https://github.com/hpdic/ElasticIVF

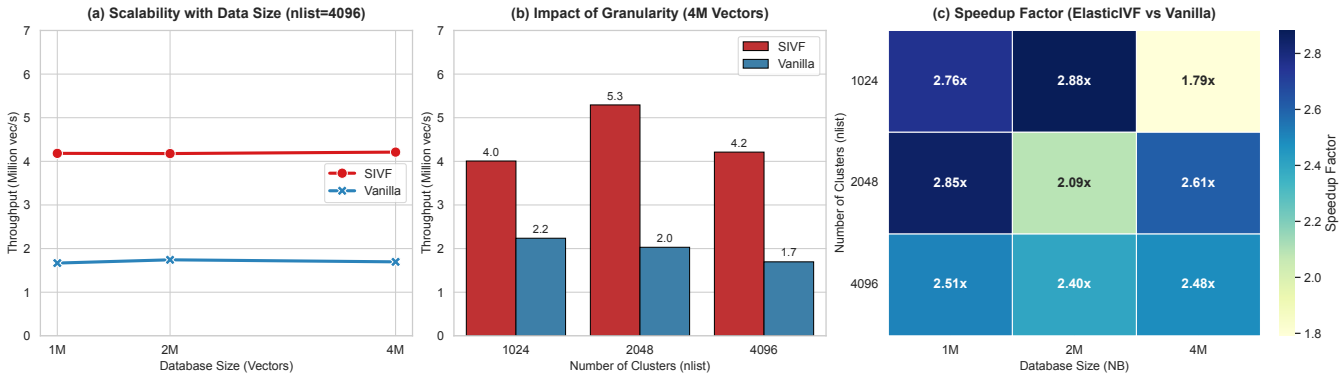

Figure 2: Performance evaluation of vector ingestion. (a) Scalability: Throughput remains remarkably stable around 4.2 million vec/s as dataset size increases from 1M to 4M (n_{list} = 4096), indicating effective handling of memory fragmentation. (b) Granularity Impact: While finer clustering granularities (n_{list} = 16384) introduce scattered write patterns, ElasticIVF maintains a 2.1×–2.5× speedup over the baseline. (c) Speedup: A heatmap showing the relative speedup of ElasticIVF over Vanilla Faiss, consistently ranging between 1.8× and 2.9×.

the baseline operates at a significantly lower tier (≈1.7M vec/s) due to contiguous memory maintenance overheads.

Impact of Clustering Granularity (Fig. 2b). We observe that ingestion throughput is sensitive to the number of clusters n_{list}:

- Peak Performance at Low n_{list}: ElasticIVF achieves its highest throughput (≈ 6.1M vec/s) at n_{list} = 1024 with 2M vectors. In this regime, the memory write patterns are more localized, reducing cache misses and maximizing the efficiency of our append kernels. Here, ElasticIVF is 2.88× faster than the baseline (≈2.1M vec/s).
- Degradation at High n_{list}: As n_{list} increases to 16384, the insertion pattern becomes highly scattered across thousands of memory slabs. Consequently, throughput for both systems decreases. However, the baseline suffers more severely from this fragmentation. ElasticIVF maintains a throughput of ≈1.3M–1.6M vec/s, achieving a 2.0× to 2.4× speedup even in this challenging high-granularity scenario. This demonstrates the resilience of the linked-slab architecture compared to the baseline's array resizing overhead.

Overall Speedup (Fig. 2c). The heatmap confirms that ElasticIVF outperforms the baseline in every tested configuration. The speedup is robust, typically ranging from 2.4× to 2.9× for most configurations. Even under the highest load (4M vectors with n_{list} = 1024), where contention for slab heads is maximized, ElasticIVF maintains a 1.79× advantage. This performance gap highlights the efficiency of our non-blocking, slab-based ingestion pipeline, making it particularly well-suited for high-throughput streaming scenarios.

4.3 SIVF Performance of Vector Search

In this section, we evaluate the query performance of ElasticIVF (SIVF) against the Vanilla Faiss GPU implementation. Theoretically, SIVF is expected to exhibit a slight degradation in search throughput compared to the baseline, as its linked-slab architecture sacrifices memory contiguity (and thus perfect memory coalescing) to enable highly efficient, fine-grained updates. Our objective is to verify that this structural overhead remains within a marginal range that does compromise overall system utility. As summarized in Figure 3, the results confirm that SIVF maintains competitive search efficiency, reaching up to 91% of the baseline performance in optimized configurations, demonstrating that the cost of supporting instant mutability is remarkably low.

Search Scalability (Fig. 3a). As the database size increases from 100,000 to 500,000 vectors (with n_{list} = 4096), SIVF throughput transitions from approximately 383k QPS to 118k QPS. The performance curve closely mirrors the baseline, confirming that our slab traversal logic maintains predictable scaling characteristics. Even at the 500k scale, SIVF provides sufficient throughput for high-concurrency production environments.

Impact of Granularity (Fig. 3b). At the 500k scale, increasing n_{list} from 1024 to 4096 significantly enhances SIVF performance from 70k to 118k QPS. This result is consistent with the principle that higher granularity reduces the average number of vectors per inverted list, thereby decreasing the total distance computations per probe. The absolute throughput remains robust across all common cluster settings.

Query Efficiency Analysis (Fig. 3c). The efficiency ratio, i.e., $QPS_{SIVF}/QPS_{Vanilla}$, reveals the effectiveness of our warp-level optimization:

- Near-Native Efficiency: At 100k vectors and n_{list} = 16384, SIVF achieves an efficiency ratio of 0.91× (295k vs. 322k QPS), nearly matching the performance of the contiguous-array-based baseline. This demonstrates that our warp-parallel search kernel effectively hides the latency associated with linked-slab traversal in sparse, high-granularity scenarios where lists are short.
- Impact of List Length: As the database size increases and inverted lists grow longer (e.g., at 500k vectors), the ratio stabilizes between 0.40× and 0.68×. This expected overhead is primarily due to increased pointer chasing between slabs, which is the physical trade-off required to enable the index's dynamic mutability.

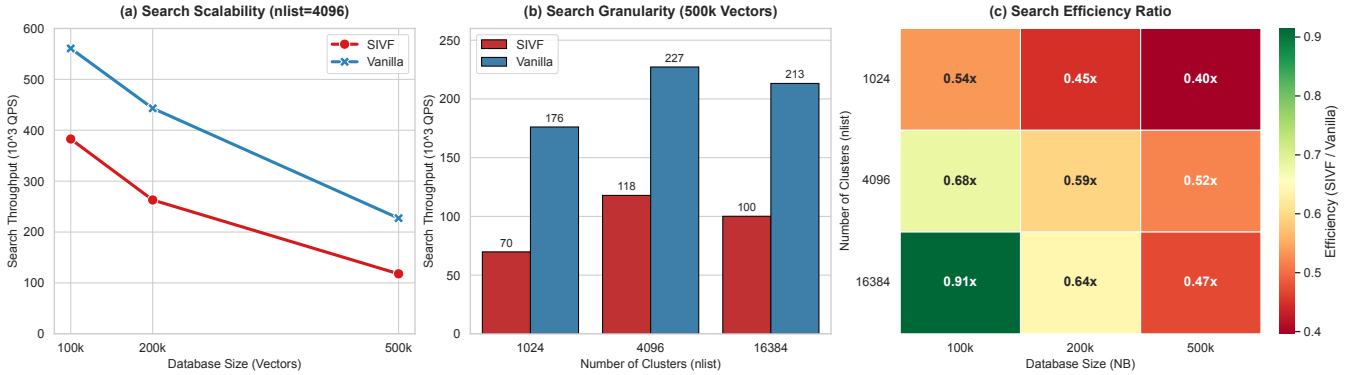

Figure 3: Performance evaluation of vector search. (a) Search Scalability: Both systems exhibit similar throughput degradation patterns as the database size grows from 100k to 500k ($n_{list} = 4096$). (b) Search Granularity: Finer clustering improves QPS by reducing the search space per probe; SIVF peaks at ≈118k QPS with $n_{list} = 4096$ (500k vectors). (c) Efficiency Ratio: Heatmap showing SIVF's query throughput relative to Vanilla Faiss, with peak efficiency reaching 0.91×.

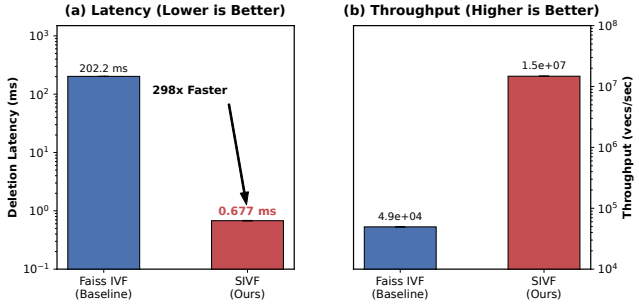

Figure 4: Performance comparison of vector deletion between Faiss IVF (Baseline) and SIVF. (a) Deletion latency (log scale), where SIVF achieves a 298.5× reduction compared to the baseline. (b) Deletion throughput (log scale), demonstrating SIVF's capacity to sustain over 14 million deletions per second.

The experimental results indicate that SIVF delivers robust search performance that is highly competitive with static index structures. By maintaining up to 91% of native GPU efficiency while providing lock-free dynamic updates, SIVF proves to be an ideal foundation for high-performance, frequently updated vector databases.

4.4 SIVF Performance of Vector Deletion

We evaluate the efficiency of the vector deletion mechanism in ElasticIVF (SIVF) by comparing it against the standard Faiss GPU IVF baseline. The experiment measures latency and throughput when removing a batch of 10,000 vectors from a populated index of 1 million 128-dimensional vectors.

Figure 4 illustrates the performance comparison between the two methods. The results indicate that the baseline Faiss implementation incurs a substantial deletion latency of approximately 202.20 ms. In stark contrast, SIVF completes the same batch deletion operation in an average of 0.68 ms. This represents a massive speedup of 298.5×. Correspondingly, deletion throughput surges from 4.9×10^4 vectors per second in the baseline to nearly 1.47×10^7 vectors per second in SIVF.

This transformative performance gain is attributed to the architectural shift in handling deletions. The baseline approach enforces contiguous memory layouts, necessitating expensive $O(N)$ memory compaction (e.g., memmove) to fill gaps left by deleted vectors. SIVF, however, leverages its GPU-resident Address Translation Table (ATT) to perform logical deletions. By mapping user IDs directly to physical slots and atomically toggling the corresponding bit in the validity bitmap, SIVF achieves $O(1)$ complexity. This mechanism effectively eliminates the overhead of physical memory reorganization, enabling ultra-low-latency updates essential for real-time streaming applications.

4.5 Real-world Datasets

To evaluate SIVF under realistic conditions, we utilized two standard large-scale benchmarks: SIFT1M ($d = 128$) and GIST1M ($d = 960$). These datasets allow us to assess system performance across varying dimensionalities, directly testing the scalability of our slab-based memory management against the contiguous memory model of the baseline (Faiss).

4.5.1 High-Throughput Ingestion. As illustrated in Figure 5, SIVF demonstrates superior ingestion scalability. On the SIFT1M dataset, SIVF achieves an ingestion rate of approximately 3.78 million vectors per second, representing a 105× speedup over the baseline (35.9k vectors/s).

This advantage persists even with the high-dimensional GIST1M dataset. While the baseline performance drops to 23k vectors/s due to the increased cost of memory reallocation and fragmentation management for large vectors, SIVF sustains a throughput of over 850k vectors/s. This 36× speedup on GIST1M confirms that our pre-allocated slab architecture effectively mitigates the overhead of dynamic resizing, allowing the system to near-saturate the GPU memory bandwidth even under heavy write loads.

4.5.2 Low-Latency Deletion. The most significant performance divergence is observed in deletion latency, shown in Figure 6. Since

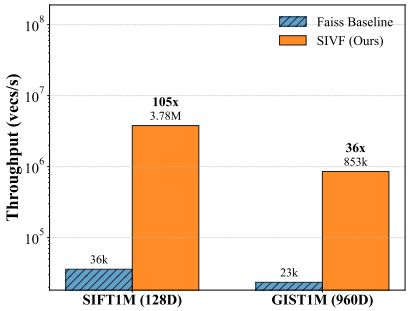

Figure 5: Ingestion throughput comparison (log scale). SIVF achieves up to 105× speedup by eliminating memory reallocation.

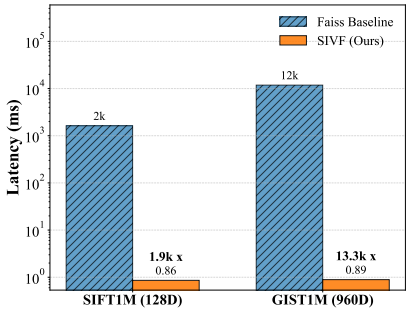

Figure 6: Deletion latency comparison (log scale). SIVF reduces latency by over 13,000× on GIST1M via in-place bitmap updates.

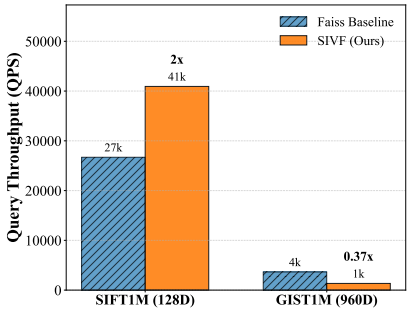

Figure 7: Search performance (QPS). While faster on SIFT1M, SIVF shows a trade-off on GIST1M due to non-contiguous memory access.

the baseline requires a full CPU-GPU roundtrip to reorganize the index, its latency is bound by PCIe bandwidth and index size. Consequently, deleting a batch of 10k vectors takes 1.6 seconds for SIFT1M and a prohibitive 11.8 seconds for GIST1M.

In contrast, SIVF performs deletion in-place via a GPU-native validity bitmap. This decouples deletion latency from both the index size and vector dimensionality. SIVF maintains a sub-millisecond latency ($\approx$ 0.89 ms) across both datasets, achieving a 1,900× improvement on SIFT1M and a staggering 13,300× improvement on GIST1M. This result validates SIVF as a viable solution for real-time streaming scenarios where data freshness is critical.

4.5.3 Search Performance and Trade-offs. Figure 7 presents the query performance (QPS) at comparable recall levels (Recall@10 $\approx$ 90%). On SIFT1M, SIVF achieves a 1.5× higher throughput than the baseline (40.9k vs. 26.7k QPS), benefiting from efficient warp-level parallelism and balanced cluster assignment.

However, on the high-dimensional GIST1M dataset, we observe a performance trade-off. SIVF achieves approximately 37% of the baseline's query throughput (1.3k vs. 3.6k QPS). This reduction is attributed to the non-contiguous memory access pattern inherent in the linked-slab design, which reduces memory coalescing efficiency when fetching large 960-dimensional vectors. We argue that this is an acceptable compromise for streaming systems, where the orders-of-magnitude improvements in mutability (ingestion and deletion) outweigh the moderate cost in raw search throughput.

5 Conclusion

In this work, we addressed the fundamental mismatch between the static design of existing GPU-accelerated ANN indices and the dynamic requirements of real-time streaming applications. Through rigorous benchmarking, we identified the "roundtrip bottleneck", i.e., the necessity of transferring index data across the PCIe bus for modification, as the primary inhibitor of low-latency eviction. To overcome this limitation, we proposed a new index architecture, namely SIVF, that relocates memory management responsibilities entirely onto the GPU. SIVF enables high-throughput and in-place mutability directly in VRAM. By implementing a slab-based memory allocator, a lock-free validity bitmap, and a GPU-resident address translation table, SIVF decouples index maintenance from the host CPU. Our evaluation on the SIFT1M and GIST1M datasets demonstrates that SIVF transforms the landscape of streaming vector search: it accelerates data deletion by up to 13,300× (sub-millisecond latency) and improves ingestion throughput by over 36× compared to the industry-standard Faiss library. These results confirm that moving memory management responsibilities from the CPU to the GPU is not only feasible but essential for the next generation of real-time neural search systems.

While SIVF successfully resolves the mutability bottleneck, several avenues remain for further optimization and expansion:

Optimizing Memory Coalescing for High Dimensions. As observed in our GIST1M experiments, the linked-slab structure introduces a trade-off in search performance due to non-contiguous memory access. Future iterations of SIVF will explore *adaptive slab sizing* and *page-aligned super-slabs* to restore memory coalescing efficiency for high-dimensional vectors ($d > 512$), aiming to bridge the gap between dynamic flexibility and raw read throughput.

Multi-GPU and Out-of-Core Scaling. Currently, SIVF operates within the memory limits of a single GPU. To support billion-scale datasets, we plan to extend the architecture to support Multi-GPU sharding via NVLink and explore unified memory techniques to leverage host RAM seamlessly. This would allow the system to handle datasets larger than aggregate VRAM capacity while maintaining the benefits of GPU-resident index management.

Metadata-Filtered Search. Real-world applications often require filtering search results based on metadata (e.g., timestamps, tags) alongside vector similarity. We intend to integrate metadata storage directly into the slab structure, allowing the search kernel to perform pre-filtering at the thread level without additional memory lookups.

Acknowledgment

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation.

References

[1] Chatzakis, M., Papakonstantinou, Y., and Palpanas, T. Darth: Declarative recall through early termination for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[2] Chung, J. W., Lin, H., and Zhao, W. Locality-sensitive indexing for graph-based approximate nearest neighbor search. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2418–2428.

[3] Deng, L., Chen, P., Zeng, X., Wang, T., Zhao, Y., and Zheng, K. Efficient data-aware distance comparison operations for high-dimensional approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 3 (Nov. 2024), 812–821.

[4] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. The faiss library.

[5] Engels, J., Landrum, B., Yu, S., Dhulipala, L., and Shun, J. Approximate nearest neighbor search with window filters. In *Forty-first International Conference on Machine Learning* (2024).

[6] Gao, J., Gou, Y., Xu, Y., Yang, Y., Long, C., and Wong, R. C.-W. Practical and asymptotically optimal quantization of high-dimensional vectors in euclidean space for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[7] Gao, J., and Long, C. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proc. ACM Manag. Data 2*, 3 (May 2024).

[8] Gong, Z., Zeng, Y., and Chen, L. Accelerating approximate nearest neighbor search in hierarchical graphs: Efficient level navigation with shortcuts. *Proc. VLDB Endow. 18*, 10 (June 2025), 3518–3530.

[9] Gou, Y., Gao, J., Xu, Y., and Long, C. Symphonyqg: Towards symphonious integration of quantization and graph for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[10] Hua, Z., Mo, Q., Yao, Z., Cui, L., Liu, X., Wang, G., Wei, Z., Liu, X., Tang, T., Liu, S., and Qu, L. Dynamically detect and fix hardness for efficient approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[11] Indyk, P., and Xu, H. Worst-case performance of popular approximate nearest neighbor search implementations: Guarantees and limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[12] Jääsaari, E., Hyvönen, V., and Roos, T. Lorann: Low-rank matrix factorization for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[13] Jiang, M., Yang, Z., Zhang, F., Hou, G., Shi, J., Zhou, W., Li, F., and Wang, S. Digra: A dynamic graph indexing for approximate nearest neighbor search with range filter. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[14] Johnson, J., Douze, M., and Jégou, H. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data 7*, 3 (2021), 535–547.

[15] Jung, S., Park, Y., Lee, H., Oh, Y. H., and Lee, J. W. Angular distance-guided neighbor selection for graph-based approximate nearest neighbor search. In *Proceedings of the ACM on Web Conference 2025* (New York, NY, USA, 2025), WWW '25, Association for Computing Machinery, p. 4014–4023.

[16] Jégou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence 33*, 1 (2011), 117–128.

[17] Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association, July 2020.

[18] Li, M., Yan, X., Lu, B., Zhang, Y., Cheng, J., and Ma, C. Attribute filtering in approximate nearest neighbor search: An in-depth experimental study. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[19] Li, Z., Ke, X., Zhu, Y., Yu, B., Zheng, B., and Gao, Y. Scalable graph indexing using gpus for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[20] Liang, A., Zhang, P., Yao, B., Chen, Z., Song, Y., and Cheng, G. Unify: Unified index for range filtered approximate nearest neighbors search. *Proc. VLDB Endow. 18*, 4 (Dec. 2024), 1118–1130.

[21] Liu, M., Zhong, S., Yang, Q., Han, Y., Liu, X., and Ma, Y. Webanns: Fast and efficient approximate nearest neighbor search in web browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (New York, NY, USA, 2025), SIGIR '25, Association for Computing Machinery, p. 2483–2492.

[22] Lu, K., Xiao, C., and Ishikawa, Y. Probabilistic routing for graph-based approximate nearest neighbor search. In *Forty-first International Conference on Machine Learning* (2024).

[23] Peng, Z., Qiao, M., Zhou, W., Li, F., and Deng, D. Dynamic range-filtering approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 10 (June 2025), 3256–3268.

[24] Qiu, R., and Tang, J. Efficient approximate nearest neighbor search via hemisphere centroids graph. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[25] Sun, P., Simcha, D., Dopson, D., Guo, R., and Kumar, S. SOAR: improved indexing for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[26] Voruganti, S., and Özsu, M. T. Mirage-anns: Mixed approach graph-based indexing for approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[27] Wang, M., Lv, L., Xu, X., Wang, Y., Yue, Q., and Ni, J. An efficient and robust framework for approximate nearest neighbor search with attribute constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (2023), A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds.

[28] Wang, Z., Zhang, J., and Hu, W. Wow: A window-to-window incremental index for range-filtering approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 6 (Dec. 2025).

[29] Wei, J., Lee, X., Liao, Z., Palpanas, T., and Peng, B. Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 1 (Feb. 2025).

[30] Xu, Q., Zhang, F., Li, C., Cao, L., Chen, Z., Zhai, J., and Du, X. Harmony: A scalable distributed vector database for high-throughput approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 4 (Sept. 2025).

[31] Yang, M., Cai, Y., and Zheng, W. CSPG: crossing sparse proximity graphs for approximate nearest neighbor search. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024* (2024), A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds.

[32] Zhang, F., Jiang, M., Hou, G., Shi, J., Fan, H., Zhou, W., Li, F., and Wang, S. Efficient dynamic indexing for range filtered approximate nearest neighbor search. *Proc. ACM Manag. Data 3*, 3 (June 2025).

[33] Zhong, X., Li, H., Jin, J., Yang, M., Chu, D., Wang, X., Shen, Z., Jia, W., Gu, G., Xie, Y., Lin, X., Shen, H. T., Song, J., and Cheng, P. Vsag: An optimized search framework for graph-based approximate nearest neighbor search. *Proc. VLDB Endow. 18*, 12 (Aug. 2025), 5017–5030.

A Appendix

A.1 Codebase Implementation and Modifications

Our implementation is built upon the Faiss library. To integrate the SIVF architecture, we introduced significant modifications to the GPU module and developed a dedicated benchmarking suite. Table 1 summarizes the actual source files added or modified in the repository based on our development history.

A.2 Sensitivity Analysis and Tuning

In addition to the main evaluation, we conducted sensitivity analyses to determine the optimal configuration for the slab size and batch processing dimensions.

Impact of Slab Capacity. We experimented with slab capacities $C \in \{16, 32, 64, 128\}$. Our results indicate that $C = 32$ provides the optimal balance. A capacity of 16 leads to excessive pointer chasing overhead (doubling the number of memory jumps), while a capacity of 128 increases the probability of warp divergence during the filtering phase. $C = 32$ aligns perfectly with the NVIDIA warp size, maximizing the efficiency of the collaborative search kernel.

Batch Size Sensitivity. Unlike CPU-based indices which typically require large batch sizes to amortize overhead, SIVF maintains high throughput even at smaller batch sizes. We observed that ingestion throughput saturates at a batch size of 256 vectors. This low saturation point validates our persistent kernel design, making SIVF highly suitable for real-time streaming where data arrives in small, frequent micro-batches.

A.3 Lessons Learned

Developing a fully dynamic, GPU-resident index presented several unique engineering challenges. We summarize the key lessons learned during the development of SIVF:

- *Implicit Synchronization Risks.* In the early stages of development, we encountered non-deterministic data corruption during high-concurrency ingestion. We traced this to the GPU's relaxed memory consistency model. Writes to a new slab were sometimes not visible to other streaming multiprocessors before the slab was linked to the list. We learned that explicit `__threadfence()` instructions are non-negotiable when implementing lock-free data structures on GPUs, even within the same kernel launch.
- *Register Pressure vs. Occupancy.* Our initial search kernel utilized complex template meta-programming to handle various metric types, which bloated the register usage per thread and limited occupancy. We found that manually unrolling loops and using simpler, distinct kernels for Inner Product and L2 distance significantly reduced register pressure, allowing more warps to be active simultaneously and hiding global memory latency more effectively.
- *The Cost of Allocators.* We initially attempted to use the built-in CUDA `malloc` for dynamic node creation. This proved disastrous for performance due to internal locking within the driver. This failure motivated the design of the custom `SlabManager`. The lesson is that for high-throughput system code, general-purpose memory allocators on the device are rarely sufficient; domain-specific, arena-based allocators are essential for performance.

Table 1: Summary of Source Files and Modifications in SIVF Project

File Path	*Description*
	Core SIVF Architecture & Memory Management
`faiss/gpu/GpuIndexSIVF.h`	Header definition for the SIVF index class, inheriting from `GpuIndex`.
`faiss/gpu/GpuIndexSIVF.cu`	Main entry point. Manages host-device synchronization and overrides standard `add`/`search` operations.
`faiss/gpu/impl/SlabManager.cuh`	Defines the device-side slab structures, arena pointers, and metadata layout.
`faiss/gpu/impl/SlabManager.cu`	Implements the Slab-based Dynamic Memory Allocator (SDMA) and memory pool management.
`faiss/gpu/impl/SIVFStructs.cuh`	Shared data structures and metadata definitions used across host and device code.
	CUDA Kernels (Ingestion, Search, Deletion)
`faiss/gpu/impl/SIVFAppend.cuh`	Device function templates for the ingestion logic.
`faiss/gpu/impl/SIVFAppend.cu`	Implements the lock-free insertion kernel, atomic slot reservation, and slab expansion logic.
`faiss/gpu/impl/SIVFSearch.cuh`	Templates for Warp-Level Collaborative Search (WLCS) and shared memory reduction.
`faiss/gpu/impl/SIVFSearch.cu`	Implements the search kernel, including distance computation and heap aggregation.
`faiss/gpu/SIVFDeletion.cu`	Implements the bitmap-based lazy eviction and deletion kernels.
	Experimental Benchmarks (C++)
`hpdic/experiment/test_sivf_add.cpp`	Benchmark driver for measuring high-throughput vector ingestion latency and speed.
`hpdic/experiment/test_sivf_search.cpp`	Benchmark driver for measuring query latency and recall (SIFT/GIST).
`hpdic/experiment/test_sivf_delete.cpp`	Benchmark driver for evaluating deletion throughput and memory reclamation.
`hpdic/experiment/sift/sift_loader.h`	Data loader utility for the SIFT1M dataset.
`hpdic/experiment/gist/gist_loader.h`	Data loader utility for the GIST1M dataset.
`hpdic/experiment/benchmark_baseline.cpp`	Baseline comparison logic for standard IVF implementations.
	Build System & Configuration
`faiss/gpu/CMakeLists.txt`	Updated CMake configuration to compile SIVF kernels and link dependencies.
`hpdic/config/c_cpp_properties.json`	VS Code configuration for IntelliSense and include paths.